

figma-implement 스킬 설명

`figma-implement` 는 완성된 Figma component 또는 제품 페이지 frame을 실제 UI 코드로 구현하는 스킬이다. 이 스킬은 디자인 시스템을 새로 만들지 않는다. `design-md` 와 `components` 가 만든 산출물을 소비해, 기존 토큰과 component contract에 맞춰 component-level 또는 page-level 구현을 수행한다.

담당 범위

- 완성된 Figma component 또는 frame 구조 수집
- Figma structure 및 asset 수집
- `DESIGN.md` 와 `tailwind.theme.css` 기준으로 style 구현
- `COMPONENTS.md` Lookup Order(+origin)에 따른 component reuse
- breakpoint별 다중 프레임을 mobile-first로 병합한 반응형 구현
- shaden primitive 재사용, 미설치 표준 shaden 설치, customized shaden 보존
- pending component 또는 route target 구현

언제 실행하는가

- 디자이너가 File A에서 구현할 Figma component를 완성했을 때
- 디자이너가 File B에서 제품 페이지 frame을 완성했을 때
- Figma URL에 `node-id` 가 포함되어 있고 구현할 target path가 정해졌을 때
- `DESIGN.md` , `tailwind.theme.css` , `COMPONENTS.md` 가 준비되어 있을 때
- Figma component 또는 page frame을 코드로 구현해야 할 때

실행하지 않는 경우

- 디자인 시스템 토큰을 처음 만들거나 갱신해야 하는 경우: `design-md` 를 실행한다.
- 새 Figma component를 contract에 등록해야 하는 경우: `components` 를 실행한다.
- `tailwind.theme.css` 가 앱에 import되지 않은 경우: 먼저 design handoff와 app integration을 완료한다.
- Figma component 또는 frame이 draft 상태라 구조가 계속 바뀌는 경우: 확정 후 실행한다.

사전 조건

다음은 먼저 확인한다.

- `DESIGN.md` 가 존재한다.
- `COMPONENTS.md` 가 존재한다.
- `tailwind.theme.css` 가 앱의 Tailwind entry stylesheet에 import되어 있다.

- Figma URL에 `node-id` 가 있다.
- target component, route, 또는 file path가 정해져 있다.

`DESIGN.md`, `COMPONENTS.md`, `tailwind.theme.css` import가 없으면 구현을 멈추고 `design-md` 또는 app integration을 먼저 처리해야 한다.

필요한 입력

필수 입력:

- Figma component 또는 frame URL with `node-id` (반응형이면 breakpoint별 URL을 함께)
- target component, route, 또는 file path
- `DESIGN.md` path
- `COMPONENTS.md` path

권장 입력:

- `tailwind.theme.css` path
- 반응형이면 mobile-first 병합 지시(base=mobile, md:=tablet, lg:=desktop)

실행 요청 예시

```
Figma URL: <file-a-component-or-file-b-frame-url-with-node-id>
Target: components/sections/PricingCard.tsx 또는 app/dashboard/page.tsx
Use DESIGN.md, tailwind.theme.css, COMPONENTS.md.
Do not create new shared components unless existing ones cannot express the design.
```

반응형(다중 프레임) 예시:

```
같은 화면의 breakpoint별 프레임이야. 하나로 병합해서 반응형으로 구현해줘.

mobile   : <file-2-frame-url-with-node-id>
tablet   : <file-2-frame-url-with-node-id>
desktop  : <file-2-frame-url-with-node-id>
Target: app/[route]/page.tsx
base=mobile, md:=tablet, lg:=desktop 차이만 덮어써줘.
Use DESIGN.md, tailwind.theme.css, COMPONENTS.md. breakpoint 값은 globals.css의 --breakpoint-* 기준.
```

사전 조건까지 명시하는 예시:

```
Figma URL: <file-2-frame-url-with-node-id>

Target: app/onboarding/page.tsx

Design docs:
- DESIGN.md
- COMPONENTS.md

Theme CSS is imported in app/globals.css.
```

실행 순서

1. 사전 조건 파일과 Tailwind theme import 상태를 확인한다.
2. Figma MCP로 component/frame structure와 assets를 수집한다(반응형이면 모든 breakpoint 프레임).
3. `COMPONENTS.md`의 Lookup Order를 origin과 함께 따른다.
4. 기존 shared component가 props, variants, slots, composition으로 표현 가능한지 확인한다.
5. shadcn 노드를 처리한다: 설치됨→재사용 / 미설치 표준→`npx shadcn add`로 설치 후 등록 / customized→`do not regenerate` 준수.
6. component-level 구현이면 matching entry를 `pending`에서 실제 path로 갱신한다.
7. page-level 구현이면 `COMPONENTS.md`에 등록된 component를 먼저 재사용한다.
8. 반응형이면 프레임들을 노드 매칭→diff(width/height/flex-direction)→mobile-first 병합한다.
9. 표현 불가능한 새 component가 필요하면 이유를 기록한다.
10. target component, route, 또는 file을 구현한다.
11. 구현된 path, `COMPONENTS.md` 갱신 내역(설치된 shadcn 경로 포함), 반응형 커버리지, blocker를 보고 한다.

출력되는 결과

주요 산출물:

산출물	설명
component-level code	pending Figma component를 지정한 shared component path로 구현
page-level code	지정한 route 또는 target page file 구현
component updates	필요한 경우 기존 component 확장 또는 route-local component 추가
asset files	Figma에서 필요한 이미지/icon asset

새 shared component가 생긴 경우 추가 산출물:

산출물	설명
new component file	재사용 가능한 shared component
creation reason	기존 component로 표현할 수 없었던 이유
<code>COMPONENTS.md</code> update 필요 항목	<code>components</code> 스킴으로 반영해야 하는 mapping

Component lookup 규칙

항상 `COMPONENTS.md`의 Lookup Order가 우선이다. 일반적인 권장 순서는 다음과 같다.

1. `COMPONENTS.md` mapped Code Path
2. `components/ui`
3. `components/sections`
4. route-local components
5. new component with reason

새 component는 다음 중 하나에 해당할 때만 만든다.

- semantic role이 다르다.
- accessibility behavior가 다르다.
- interaction model이 다르다.
- 기존 props, variants, slots, composition으로 표현할 수 없다.

스타일 구현 규칙

- color, typography, spacing, radius는 `DESIGN.md` 또는 `tailwind.theme.css` token을 사용한다.
- hardcoded hex value를 쓰지 않는다.
- hardcoded font family를 쓰지 않는다.
- pixel value는 layout 구현상 불가피한 경우라도 token과의 관계를 확인한다.
- breakpoint 값은 임의의 px이 아니라 `DESIGN.md ## Layout / globals.css --breakpoint-*`를 따른다.
- Figma와 다르게 보이는 값을 임의로 감각 보정하지 않는다.
- token이 누락되어 있으면 구현 중 임의의 token을 만들기보다 `design-md` 갱신 필요성을 보고한다.

반응형 구현 규칙

Figma 프레임은 고정 크기 스냅샷이고, breakpoint 간 레이아웃 재배치는 별도 프레임에 산다. 에이전트는 여러 프레임이 같은 화면인지 자동으로 알지 못하므로, 병합에는 3조건이 필요하다.

1. 다중 프레임 동시 전달 — 같은 target의 breakpoint별 프레임을 한 번에 준다.
2. 정체성 신호 — 프레임들이 같은 File A instance(같은 component key/이름) 또는 일관된 레이어 이름을 써서 노드를 대응시킬 수 있어야 한다.

3. 병합 지시 — 이들이 한 화면의 breakpoint 변형임을 명시한다.

조건이 갖춰지면 mobile-first로 병합한다.

- 각 프레임의 node tree를 읽어 대응 노드를 매칭한다.
- 노드별 반응형 속성(width, height, `layoutMode` (flex direction), wrap, gap, padding, 정렬, visibility)을 diff한다.
- 가장 작은 breakpoint를 base로, 큰 breakpoint에서 차이만 접두사로 덮어쓴다.
 - `layoutMode: VERTICAL` → `flex-col`, `HORIZONTAL` → `flex-row`. 예: mobile VERTICAL, desktop HORIZONTAL → `flex-col lg:flex-row`.
 - `itemSpacing` → `gap-*`, sizing mode → `w-full` / `w-fit` / 고정 `w-*` 에 접두사.
- token-breakpoint 접두사 유틸만 쓴다. hardcoded `min-width: 900px` 같은 값 금지.

단일 프레임만 온 반응형 화면은 받은 그대로 구현하되, 결과가 해당 breakpoint에 고정됨과 추가 프레임이 필요함을 보고한다. breakpoint 동작을 제어내지 않는다.

shadcn/ui 처리 규칙

`COMPONENTS.md`의 origin(`shadcn` / `shadcn (customized)` / `custom`)을 노드별로 따른다.

- **shadcn**, 설치됨 → 기존 `components/ui` primitive를 props/variant로 재사용한다. 처음부터 다시 스타일링하지 않는다.
- **shadcn**, 미설치 → 손으로 짜지 말고 `npx shadcn@latest add <name>` 로 설치한 뒤 `COMPONENTS.md`를 `pending` → 실제 경로로 갱신하고 사용한다.
- **shadcn (customized)** → `do not regenerate` 를 준수한다. 재설치하지 말고 기존 커스텀 컴포넌트와 그 로컬 variant/props API를 그대로 쓴다. 추가 커스텀이 필요하면 기존 파일을 수정하고 `COMPONENTS.md` note를 갱신한다.
- **custom** → 일반 Lookup Order를 따른다(있으면 재사용/확장, `pending` 이면 구현, 표현 불가할 때만 이유 기록 후 생성).

Figma component는 코드가 아니라 디자인 명세다. "디자이너 컴포넌트를 쓴다"는 그 structure/variant를 읽어 기존 코드에 매핑하거나 코드로 구현하는 것이지 Figma 노드를 직접 import하는 게 아니다.

다른 스킴과 연계하는 방법

`design-md`와의 관계:

- `figma-implement`는 `design-md`의 산출물을 전제로 한다.
- `DESIGN.md`는 디자인 판단 기준이다.
- `tailwind.theme.css`는 실제 코드에서 사용할 Tailwind token namespace다.
- token이 부족하거나 Figma와 token이 다르면 `design-md`를 다시 실행해야 한다.

`components`와의 관계:

- `COMPONENTS.md`는 component reuse 기준이다.
- `Code Path`가 확인된 component는 우선 재사용한다.

- `pending` component가 구현에 필요하면 먼저 component 구현 또는 mapping 확정이 필요하다.
- 새 shared component를 만들면 `components` 스킴로 `COMPONENTS.md`를 갱신해야 한다.

실패 또는 중단 기준

- Figma URL에 `node-id`가 없으면 구현 대상을 확정할 수 없다.
- `DESIGN.md`가 없으면 중단한다.
- `COMPONENTS.md`가 없으면 중단한다.
- `tailwind.theme.css`가 app stylesheet에 import되지 않았으면 중단한다.

운영상 주의점

- 이 스킴은 "디자인을 보고 새로 창작"하는 스킴이 아니라 "합의된 handoff artifacts를 소비"하는 스킴이다.
- 기존 component로 표현 가능한 UI를 새 shared component로 만들면 안 된다.
- 새 component가 필요할 때는 구현보다 먼저 이유가 있어야 한다.
- 표준 shaden primitive는 hand-write하지 말고 CLI로 설치한다. customized shaden은 절대 재생성하지 않는다.
- 반응형 병합은 다중 프레임·정체성 신호·병합 지시가 모두 있을 때만 수행하고, 그 외에는 단일 breakpoint 고정임을 보고한다.